Programme Code: TU653, TU757, TU860, TU758, TU863

Module Code: COMP H2014

CRN: 17109

TECHNOLOGICAL UNIVERSITY DUBLIN

BLANCHARDSTOWN CAMPUS

TU653 - Higher Certificate in Science in Computing in information Technology

TU757 - Bachelor of Science in Computing in Information Technology

TU860 - Bachelor of Science (Honours) in Computing

TU758 - Bachelor of Science in Computing in Digital Forensics & Cyber Security

TU863 - Bachelor of Science (Honours) in Computing in Digital Forensics & Cyber Security

Year 2

_____

SEMESTER 2

EXAMINATIONS 2023/24

_____

Operating Systems

Internal Examiner: Dr. Kevin Farrell

External Examiner: Mr. Tony Davitt

Exam Duration: 2 hours

Instructions to candidates:

1.	To ensure that you take the correct examination, please check that the module and programme which you are following is listed above.
2.	Question 1 in Section A is **COMPULSORY**. Candidates should attempt **ALL** parts of Question 1 in Section A, and **ANY two** other questions in Section B.
3.	Where a question asks you to use diagrams, to obtain full marks, you **MUST** provide diagrams.
4.	Where a question asks you to use code, to obtain full marks, you **MUST** provide code.
5.	This paper is worth **100** marks. Question 1 is worth **40** marks. All other questions are worth **30** marks each.
6.	Please make sure you write the question numbers you answer in the boxes on the front of your answer book.

DO NOT TURN OVER THIS PAGE UNTIL YOU ARE TOLD TO DO SO

SECTION A: COMPULSORY QUESTION

Question 1: Answer *ALL* parts of this question *(4 marks each part)*
Please write the number of this question on the front of your answer book

a) Describe the *Bounded-Buffer Producer-Consumer Problem*, detailing why a synchronisation solution is necessary.

b) Provide **two** commands that can be used to view information about processes running on a Linux system. Give a **one** line description in each case as to what *type* of information each command gives.

c) With reference to the concept of a Process Control Block, in a *multi-threaded* process why is the time taken, to perform a *context-switch* between **two** individual threads in the same process, shorter than the time taken to perform a context-switch between **two** *single-threaded* processes?

d) In relation to hardware solutions to the Critical Section Problem, explain what *disabling interrupts* is, and how it potentially solves the Critical Section Problem in kernel code, run on a single processor machine.

e) In the Dining Philosophers problem, which you studied during the semester, what condition, necessary for *deadlock* to occur, is eliminated in the *Lefty-Righty* solution and the *Four Chairs* solution?

f) Give the definition of a *socket*. Give an example.

g) Distinguish between *physical* and *virtual* resources in an Operating System. Give **one** example of each.

This question is continued on the next page...

h) A file in a Linux Operating System has the following permissions:

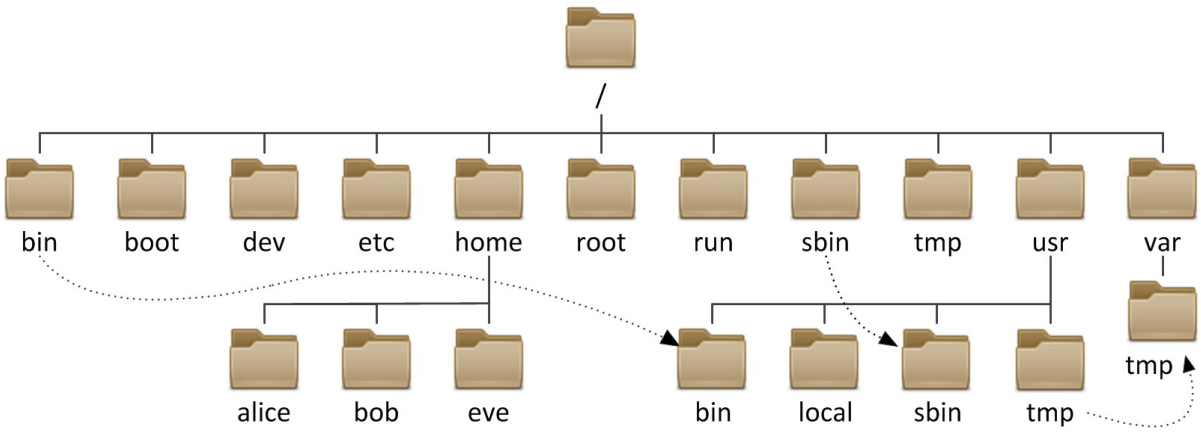
```
rw- r-- r--
```

What is the *octal* representation of these permissions?

i) Draw a diagram of the One-to-One model of multithreading and briefly describe it.

j) The diagram below shows the layout of the Linux *Filesystem*. Consider the case of the user **`eve`**, whose Current Working Directory is her home directory **`/home/eve`**. In each case below, provide the *single* command, with appropriate arguments, that **`eve`** needs to type in order to change directory from **`/home/eve`** to **`/usr/local`** using:

1. A relative path.
2. An absolute path.



(40 Marks Total)

Please now write the number of this question in the box on the front of your answer book.

SECTION B: ANSWER ANY *TWO* QUESTIONS

Question 2

If you answer this question please write the number on the front of your answer book

a) What is a thread, and what is meant by a multi-threaded process?

(4 marks)

b) Explain, with the aid of a diagram, how a *multi-threaded* process with **four** threads could run potentially **four** times faster compared to a single-threaded process of the same program, when both are run on the same quad-core machine.

(6 marks)

c) Describe the *modular kernel* methodology of operating systems design. Briefly describe **one** benefit of this approach and **one** way in which it differs from the *layered* operating systems design methodology.

(6 marks)

d) Using the following headings, describe the **two** modes of execution of processes in an Operating System (OS), namely *user-mode* and *kernel-mode*:

i. What are the features of each mode?
ii. What are the benefits of having the **two** modes?
iii. What would cause the OS to switch a process from *user-mode* to *kernel-mode*?
iv. How does the switch from *user-mode* to *kernel-mode t*ake place; i.e. what is the mechanism? Use a diagram to aid your answer for this part.

(14 marks)

(30 Marks Total)

Did you answer this question? If so, please now write the number of this question in the box on the front of your answer book.

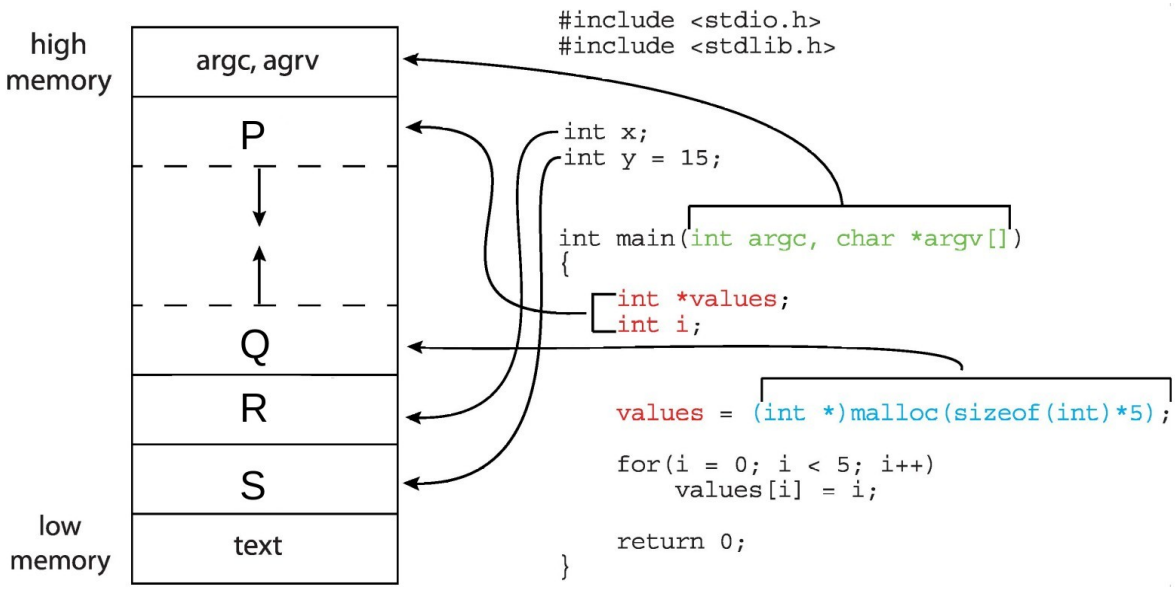
Question 3

If you answer this question please write the number on the front of your answer book

a) Distinguish between a *Program* and a *Process* in an Operating System.

(2 marks)

b) The diagram below shows the memory layout of a *Process* of a C Program. Write, in your answer book, the *names* of the areas of memory labelled, P, Q, R, and S, along with a **one-line** explanation of the type of instructions or data held in that memory area.



(8 marks)

c) In relation to a *Process Control Block (PCB)*, explain:

i. What is a PCB?

ii. Briefly describe **four** items it contains.

iii. What role does a PCB play in a context switch?

(8 marks)

This question is continued on the next page...

d) In an Operating System, **five** different processes arrive at time 0 in the order: Process #1, #2, #3, #4, #5. The *low level (short-term)* scheduling algorithm then orders the queue as follows:

Process #	Estimated Run Time (s)	Waiting Time (s)
3	1	0
1	2	1
4	10	3
5	21	13
2	100	34

i. In your opinion, what is the scheduling algorithm used here? **Hints**: It is a *non-pre-emptive* algorithm. Give reason(s) for your answer.
ii. Calculate the *average wait-time* for the algorithm using the data in the table above.
iii. Using the same Process information as above, create a similar table for the *First-Come First-Served* scheduling algorithm.
iv. Using your table from part iii, calculate the *average wait-time* for the First-Come First Served algorithm.
v. Using the **two** values for the *average wait-time*, which is the better algorithm? Give reason(s) for your answer.

(12 marks)

(30 Marks Total)

Did you answer this question? If so, please now write the number of this question in the box on the front of your answer book.

Question 4

If you answer this question please write the number on the front of your answer book

a) What is a *race condition*?

(2 marks)

b) List and describe the **three** conditions necessary to solve the *Critical Section Problem*. Also, briefly list any assumption(s) about the relative speed of processes associated with the **three** conditions.

(7 marks)

c) Provide the code for the *Test_and_Set Instruction.* Explain a property of the instruction. Also, explain how the code works.

(8 marks)

d) The code below shows how the *Test_and_Set Instruction* can be used to solve the *Critical Section Problem*. Referring to the execution of **two** or more processes sharing variables, explain how the code satisfies the *mutual exclusion* condition. Your answer must state what the initial conditions are.

```
do {
        while (test_and_set(&lock)= = TRUE)
                ;

        /* critical section */

        lock = false;

        /* remainder section */

} while (true);
```

(10 marks)

e) Briefly explain in what *type* of code would the *Test_and_Set Instruction* be generally used. Give a reason for your answer.

(3 marks)

(30 Marks Total)

Did you answer this question? If so, please now write the number of this question in the box on the front of your answer book.